Université Sultan Moulay Slimane
Ecole Supérieure de Technologie
- Beni Mellal -

STRUCTURES DE DONNÉES

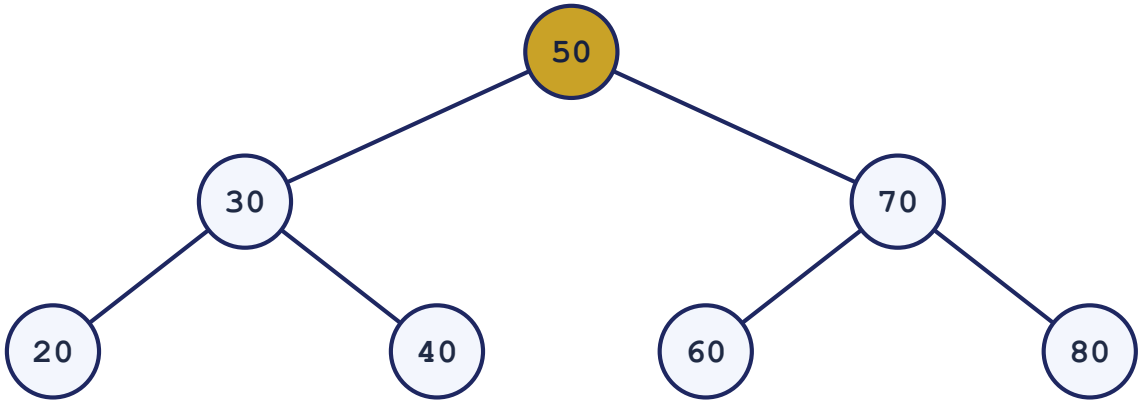
Chapitre 7 : Les Arbres Binaires de Recherche (ABR)

Filière : DUT GI
Semestre : S3
Année Universitaire : 2026/2027

Pr. M. OUTANOUTE

L'invariant d'ordre

Dans l'arbre de l'exercice de fin du Chapitre 6, le parcours infixe donnait les valeurs triées — et ce n'était pas un hasard. Ce chapitre formalise l'invariant qui explique cette remarque.



Arbre binaire de recherche (***ABR***) : pour chaque nœud, tout le sous-arbre gauche contient des valeurs plus petites, tout le sous-arbre droit des valeurs plus grandes — en tout point de l'arbre.

La recherche d’une valeur dans un arbre quelconque

```
int rechercherArbre(Noeud *n, int valeur) {
    if (n == NULL) return 0;
    if (n->valeur == valeur) return 1;
    return rechercherArbre(n->gauche, valeur) || rechercherArbre(n->droit, valeur);
}
```

Sans règle d'ordre, il faut potentiellement explorer les deux sous-arbres — contrairement à la recherche dans un ***arbre binaire de recherche***, qui n'explorera qu'un seul côté à chaque étape.



Ce que l'invariant permet : *la recherche dirigée*

```
// Recherche dans un arbre quelconque :
return rechercherArbre(n->gauche, valeur) || rechercherArbre(n->droit, valeur);
```

Recherche dans un ***ABR*** : un seul côté, dirigé par la comparaison.

Le gain concret :

Une recherche dans un *ABR* ne descend jamais que d'un seul côté à chaque étape

— contre les deux sous-arbres explorés systématiquement dans le cas d’un arbre quelconque.

Trois opérations, un vrai défi

Chercher : *rechercherABR*.

Ajouter : *insererABR*.

Supprimer : *supprimerABR*, trois cas — plus tard. Le vrai défi de ce chapitre.

Le fil narratif : le registre, trié

Le registre des inscrits du Chapitre 2, repris
— désormais trié par numéro d'étudiant, grâce à l'invariant.

Récapitulatif

- ABR : pour chaque nœud, sous-arbre gauche plus petit, sous-arbre droit plus grand.
- C'est cet invariant qui explique le parcours infixe trié de l’exercice du Chapitre 6.
- Le gain : une recherche dirigée, un seul côté exploré à chaque étape.
- Trois opérations : ***chercher***, ***ajouter*** et ***supprimer*** (le vrai défi).
- Fil narratif : le registre des inscrits, désormais trié par numéro.

Le nœud, inchangé

```
typedef struct noeud {
    int valeur;
    struct noeud *gauche;
    struct noeud *droit;
} Noeud;

Noeud *racine = NULL;
```

Même nœud qu'au Chapitre 6 — seul l'invariant est nouveau, pas la structure.

rechercherABR

1) Cas de base : si le nœud est NULL, la valeur n'est pas dans l'arbre — retourner 0.
2) Si la valeur du nœud correspond, retourner 1.
3) Si la valeur cherchée est plus petite, chercher dans le sous-arbre gauche uniquement.
4) Sinon, chercher dans le sous-arbre droit uniquement.

```
int rechercherABR(Noeud *n, int valeur) {
    if (n == NULL) return 0;
    if (valeur == n->valeur) return 1;
    if (valeur < n->valeur) {
        return rechercherABR(n->gauche, valeur);
    }
    return rechercherABR(n->droit, valeur);
}
```



insererABR : un nouvel idiome

Insérer doit ajouter la valeur à la bonne place, sans violer l'invariant.
Cette fois, la fonction va retourner un nœud — idiome nouveau.

1) Cas de base : si le nœud est NULL, on a trouvé la bonne place — créer et retourner un nouveau nœud.
2) Si la valeur est plus petite, insérer dans le sous-arbre gauche, et réaffecter ce sous-arbre au résultat.
3) Si la valeur est plus grande, insérer dans le sous-arbre droit, et le réaffecter de même.
4) Retourner le nœud (inchangé, sauf à l'étape 1).

insererABR — le code

```
Noeud* insererABR(Noeud *n, int valeur) {
    if (n == NULL) { return creerNoeud(valeur); }
    if (valeur < n->valeur) {
        n->gauche = insererABR(n->gauche, valeur);
    } else if (valeur > n->valeur) {
        n->droit = insererABR(n->droit, valeur);
    }
    return n;
}
```

Pourquoi retourner un nœud ? Insérer peut créer un nouveau nœud là où il n'y avait que NULL — il doit être rattaché à son parent. Réaffecter propage ce rattachement à chaque niveau.

L'appel initial : *racine = insererABR(racine, valeur);* — racine elle-même doit être réaffectée.

Récapitulatif

- **rechercherABR** : un seul appel récursif par branche, dirigé par la comparaison.
- **insererABR** : nouvel idiome — la fonction retourne un nœud, l'appelant réaffecte.
- Nécessaire car l'insertion peut créer un nœud là où il n'y avait que NULL.
- L'appel initial suit le même principe : ***racine = insererABR(racine, valeur);***



Pourquoi la suppression est différente

Le nœud à retirer peut avoir des enfants, qui doivent rester rattachés à l'arbre — et l'invariant d'ordre doit rester respecté après l'opération.

Trois cas se présentent :

Cas 1 : le nœud est une feuille.

Cas 2 : le nœud a un seul enfant.

Cas 3 : le nœud a deux enfants — le cas riche.

Cas 1 — le nœud est une feuille

1) Si le nœud n'a ni enfant gauche ni enfant droit, le libérer, et retourner NULL à sa place.

```
if (n->gauche == NULL && n->droit == NULL) {
    free(n);
    return NULL;
}
```

Le cas simple — rien à rattacher, la place redevient NULL.

Cas 2 — un seul enfant

1) Si le nœud n'a pas d'enfant gauche, mémoriser son enfant droit, libérer le nœud, et retourner cet enfant.
2) Symétriquement, si le nœud n'a pas d'enfant droit, faire de même avec l'enfant gauche.

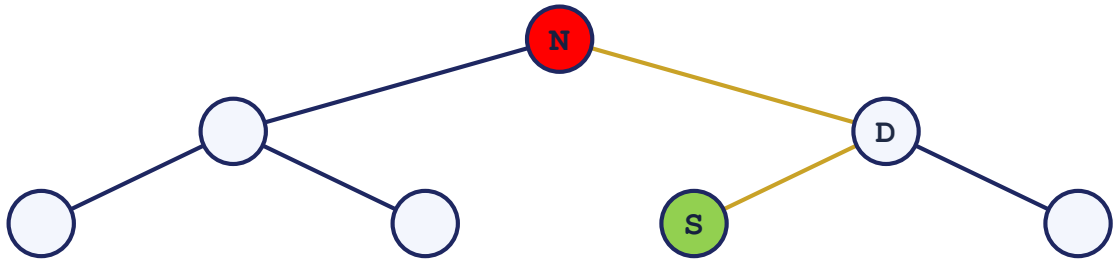
```
if (n->gauche == NULL) {
    Noeud *temp = n->droit;
    free(n);
    return temp;
}
if (n->droit == NULL) {
    Noeud *temp = n->gauche;
    free(n);
    return temp;
}
```

L'enfant restant prend directement la place — l'invariant reste respecté, il était déjà dans la bonne zone.

Cas 3 — deux enfants, le successeur

Le nœud a deux enfants — impossible de le retirer sans compromettre l'arbre.
La solution : remplacer sa valeur par celle du nœud qui lui succède immédiatement dans l'ordre.

Le successeur (dans l'ordre infixe) : la plus petite valeur du sous-arbre droit — le nœud le plus à gauche possible, en partant de ce sous-arbre droit.



***N** = nœud à supprimer, **D** = son enfant droit , **S** = successeur (le plus à gauche sous D)*

trouverMin — la sous-routine

1) Partir du nœud donné.
2) Tant qu'il existe un enfant gauche, avancer vers cet enfant gauche.
3) Retourner le nœud atteint — il n'a pas d'enfant gauche.

```
Noeud* trouverMin(Noeud *n) {
    while (n->gauche != NULL) {
        n = n->gauche;
    }
    return n;
}
```

Cas 3 — avec la sous-routine

1) Trouver le successeur : le minimum du sous-arbre droit du nœud à supprimer.
2) Copier la valeur du successeur dans le nœud à supprimer.
3) Supprimer, récursivement, le successeur de sa position d'origine, dans le sous-arbre droit.

```
Noeud *successeur = trouverMin(n->droit);
n->valeur = successeur->valeur;
n->droit = supprimerABR(n->droit, successeur->valeur);
```

Pourquoi c'est sûr : le successeur, le plus à gauche de son sous-arbre, a au plus un enfant (jamais de gauche).

Sa suppression retombe toujours sur le cas 1 ou 2 — jamais de nouveau sur le cas 3.

supprimerABR — l'assemblage complet

```
Noeud* supprimerABR(Noeud *n, int valeur) {
    if (n == NULL) return NULL;
    if (valeur < n->valeur) n->gauche = supprimerABR(n->gauche, valeur);
    else if (valeur > n->valeur) n->droit = supprimerABR(n->droit, valeur);
    else {
        if (n->gauche == NULL && n->droit == NULL) { free(n); return NULL; }
        if (n->gauche == NULL) { Noeud *t = n->droit; free(n); return t; }
        if (n->droit == NULL) { Noeud *t = n->gauche; free(n); return t; }
        Noeud *successeur = trouverMin(n->droit);
        n->valeur = successeur->valeur;
        n->droit = supprimerABR(n->droit, successeur->valeur);
    }
    return n;
}
```

Récapitulatif

- Cas 1 (feuille) : libérer, retourner NULL.
- Cas 2 (un enfant) : l'enfant restant prend directement la place.
- Cas 3 (deux enfants) : copier la valeur du successeur, puis le supprimer récursivement.
- Successeur = minimum du sous-arbre droit — a au plus un enfant, jamais de gauche.
- Même idiome que ***insererABR*** : retourner et réaffecter.

Exercice d’application

Le registre des inscrits est représenté par un ABR, trié par numéro, racine étant une variable globale.

1) Écrire ***inscrireEtudiant*** et ***estInscrit***.

2) Dérouler l'inscription de 7 étudiants : 205, 150, 260, 120, 180, 240, 280.

3) Écrire ***radierEtudiant***. Dérouler la radiation de 205 — expliquer.

Questions 1 et 3 — le code

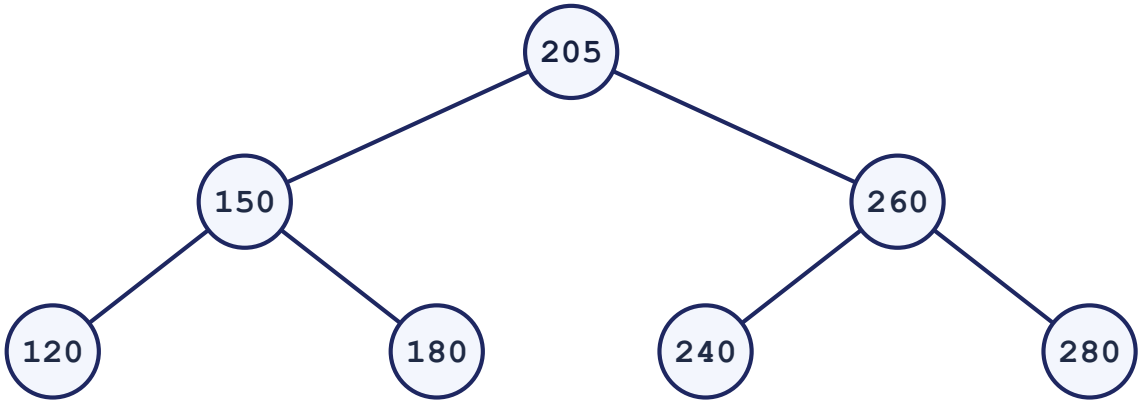
```
void inscrireEtudiant(int numero) {
    racine = insererABR(racine, numero);
}

int estInscrit(int numero) {
    return rechercherABR(racine, numero);
}

void radierEtudiant(int numero) {
    racine = supprimerABR(racine, numero);
}
```

Trois fonctions minces, qui réutilisent le code déjà connu — rien de nouveau à inventer.

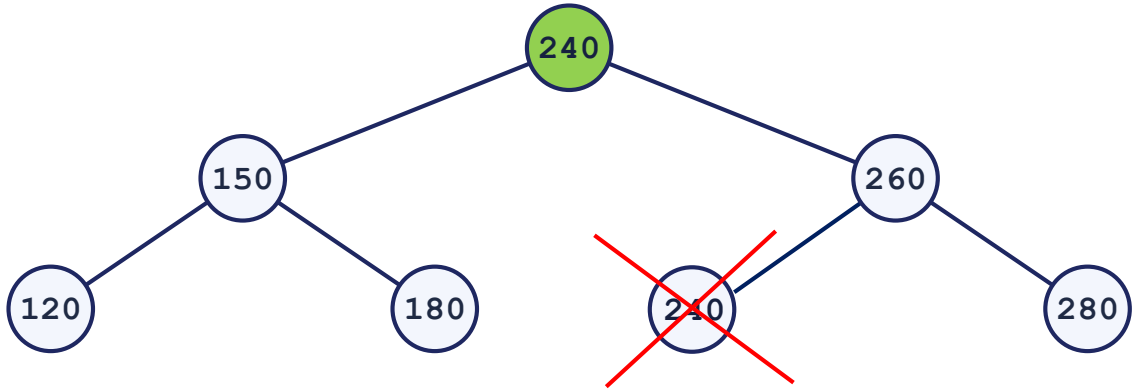
Question 2 — l'arbre construit



Parcours infixe : 120 150 180 205 240 260 280 — trié, grâce à l'invariant.

Question 3 — la radiation de 205

205, la racine, a deux enfants (150 et 260) — le cas 3.



Successeur de 205 = minimum du sous-arbre droit (260) = 240.

240 remplace 205 ; l'ancien 240 est supprimé (cas 1, une feuille).